

Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б08-мм

Реализация алгоритмов верификации вероятностных приближённых ограничений в “Desbordante”

СЕНИЧЕНКОВ Пётр Ильич

Отчёт по учебной практике
в форме “Производственное задание”

Научный руководитель:
кфмн ассистент кафедры ИАС Чернышев Г. А.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	5
2. Предварительные сведения	6
2.1. Основные определения	6
2.2. Примеры	8
2.3. Алгоритм РАС-Man	10
3. Описание решения	12
4. Апробация	18
5. Заключение	24
Список литературы	25

Введение

Профилирование данных — это процесс, направленный на получение метаданных, то есть данных о данных [1]. Метаданные могут быть простыми, такими как количество пустых значений и сумма значений в столбцах. Более сложные метаданные представляют собой различные закономерности, часто включающие несколько столбцов в таблице.

Примерами таких закономерностей являются функциональная зависимость (Functional Dependency, FD) и уникальная комбинация столбцов (Unique Column Combination, UCC).

Часто данные, полученные в ходе эксперимента, содержат неточности вследствие погрешности измерений, ошибок при сборе, передаче и агрегации данных, и так далее. В таком случае рассматривают приближённый или вероятностный вариант зависимости. Например, функциональная зависимость $X \rightarrow Y$ означает, что значения в правой части однозначно определяют значения в левой части, то есть каждому значению из столбца X соответствует ровно одно значение из столбца Y . Вероятностная функциональная зависимость (Probabilistic Functional Dependency, PFD) означает, что условие функциональной зависимости выполняется с высокой вероятностью, то есть для большого числа строк в таблице. Приближённым (в смысле небольших погрешностей в данных) вариантом FD является метрическая функциональная зависимость (Metric Functional Dependency, MFD), которая означает, что одинаковым значениям в левой части соответствуют близкие значения в правой части.

Дальнейшим обобщением являются вероятностные приближённые ограничения (Probabilistic Approximate Constraints, PAC), которые объединяют вероятностный и приближённый вариант зависимости. В предыдущей работе [7] были рассмотрены вероятностные приближённые ограничения по области (Domain PAC), которые означают, что большинство значений в столбце находятся достаточно близко к некоторой области. В данной работе рассматриваются функциональные вероятностные приближённые ограничения (Function Dependency PAC, FD PAC) и

вероятностные приближённые ограничения по уникальной комбинации столбцов (Unique Column Combination PAC, UCC PAC). FD PAC представляют собой обобщение FD и означают, что, если значения в левой части близки, то и значения в правой части будут близки с достаточно высокой вероятностью. UCC PAC являются обобщением UCC и означают, что большинство значений достаточно далеки друг от друга.

Извлечение сложных метаданных из данных требует применения специальных инструментов. Одним из таких инструментов является DESBORDANTE — наукоёмкий профилировщик данных [2] с открытым исходным кодом¹. Данная работа направлена на добавление в проект возможности верификации задаваемых пользователем FD PAC и UCC PAC.

¹<https://github.com/Desbordante/desbordante-core>

1. Постановка задачи

Целью работы является разработка и внедрение алгоритмов верификации FD PAC и USS PAC в DESBORDANTE. Для достижения цели были поставлены следующие задачи:

1. реализовать алгоритмы верификации FD PAC и USS PAC на C++;
2. разработать модульные тесты для проверки корректности работы алгоритмов и соответствующий набор данных для тестов;
3. реализовать Python-привязки к алгоритмам верификации;
4. создать примеры использования;
5. провести апробацию и сравнение алгоритмов верификации PAC.

2. Предварительные сведения

Обзор предметной области был проведён в предыдущей работе [7]. Здесь приведём основные определения, простые примеры и краткое описание алгоритма PAC-Man.

2.1. Основные определения

Все определения, приведённые в данном разделе (кроме опр. 4), взяты из работы F. Korn и др. [5]. При этом, использованы традиционные для данной области обозначения.

Пусть $R(U)$ — схема отношения (в терминах реляционной модели данных), где R — имя отношения, а $U = \{ A_1, \dots, A_n \}$ — множество атрибутов. Отношение r над $R(U)$ представляет собой конечный набор кортежей. Область определения атрибута A обозначим $\text{dom}(A)$. Значение кортежа t на атрибуте A будем обозначать $t[A]$.

Для множества атрибутов $X = \{ A_1, \dots, A_n \}$ обозначим $\text{dom}(X) = \text{dom}(A_1) \times \dots \times \text{dom}(A_n)$. Значением кортежа t на множестве атрибутов $X = \{ A_1, \dots, A_n \}$ будем называть набор $(t[A_1], \dots, t[A_n])$. Значение кортежа t на множестве атрибутов X обозначим $t[X]$.

Определение 1 (Domain PAC). Пусть X — множество атрибутов, на множестве $\text{dom}(X)$ задана метрика ρ , $D \subset \text{dom}(X)$ — линейно упорядоченная область.

Будем говорить, что выполняется *вероятностное приближённое ограничение по области D* (Domain Probabilistic Approximate Constraint, Domain PAC), если значения $x \in \text{dom}(X)$ находятся на расстоянии не более ε от D с вероятностью не меньше δ :

$$\Pr(\rho(x, D) \leq \varepsilon) \geq \delta$$

◁

Определение 2 (FD PAC). Пусть X, Y — множества атрибутов, на множествах $\text{dom}(X), \text{dom}(Y)$ заданы метрики ρ_X и ρ_Y соответственно.

Функциональное вероятностное приближённое ограничение (Func-

tional Dependency Probabilistic Approximate Constraint, FD PAC) $X \rightarrow Y$ означает, что, если значения в левой части различаются не более, чем на Δ_l , то и соответствующие им значения в правой части будут различаться не более, чем на ε_l с вероятностью не меньше δ , то есть, если

$$\rho_X(t_i[A_l], t_j[A_l]) \leq \Delta_l \quad \forall A_l \in X,$$

то

$$\Pr(\rho_Y(t_i[B_l], t_j[B_l]) \leq \varepsilon_l) \geq \delta \quad \forall B_l \in Y.$$

◁

FD PAC являются обобщением функциональных зависимостей (Functional Dependency, FD).

Определение 3 (UK PAC). Пусть $X = \{ A_1, \dots, A_n \}$ — множество атрибутов, на $\text{dom}(A_l)$ для каждого атрибута $A_l \in X$ задана метрика ρ_l .

Вероятностное ограничение по уникальному ключу (Unique Key Probabilistic Approximate Constraint, UK PAC) означает, что вероятность, что существует несколько кортежей с близкими значениями ключевых атрибутов (значения различаются не более, чем на ε_l), не превосходит δ_l :

$$\Pr(\rho_l(t_i[A_l], t_j[A_l]) \leq \varepsilon_l) \geq \delta_l \quad \forall A_l \in X$$

◁

UK PAC задаёт ограничение для каждого атрибута в отдельности и может быть записано в виде конъюнкции ограничений по одиночным атрибутам:

$$\text{UK PAC}_{\varepsilon_1, \dots, \varepsilon_n}^{\delta_1, \dots, \delta_n}(\{ A_1, \dots, A_n \}) = \text{UK PAC}_{\varepsilon_1}^{\delta_1}(A_1) \wedge \dots \wedge \text{UK PAC}_{\varepsilon_n}^{\delta_n}(A_n)$$

Значит, достаточно определить UK PAC только для одного атрибута. При этом алгоритм PAC-Map подходит только для верификации UK PAC на одном атрибуте (см. раздел 2.3), и UK PAC на нескольких атрибутах пришлось бы верифицировать как набор ограничений на одиночных атрибутах.

Введём также определение ограничения для комбинации атрибутов, более подходящего для верификации при помощи алгоритма PAC-Map.

Определение 4 (UCC PAC). *Вероятностное приближённое ограничение по уникальной комбинации столбцов (Unique Column Combination Probabilistic Approximate Constraint, UCC PAC)* означает, что вероятность, что существует несколько кортежей с близким значением ключа (значения различаются не более, чем на ε), не превосходит δ :

$$\Pr(\rho(t_i[X], t_j[X]) \leq \varepsilon) \geq \delta$$

◁

Заметим, что для одиночного атрибута UK PAC и UCC PAC совпадают. При этом, UCC PAC на нескольких атрибутах нельзя выразить как набор ограничений на одиночных атрибутах. Таким образом, UCC PAC является обобщением UK PAC.

Вследствие приведённых соображений UK PAC в дальнейшем рассматривать не будем. В предыдущей работе можно найти подробное сравнение UK PAC и UCC PAC на примере отношения с двумя атрибутами.

Для удобства разделим параметры PAC на две группы.

Определение 5. Параметры ε и δ (а также множества $\{\varepsilon_l\}$ и $\{\delta_l\}$ для FD PAC и UK PAC) будем называть *вероятностными параметрами PAC*. Будем говорить, что все остальные параметры (наборы атрибутов, метрики на доменах атрибутов, домен для Domain PAC и множество $\{\Delta_l\}$ для FD PAC) задают *правило* или *шаблон PAC*. ◁

2.2. Примеры

Рассмотрим примеры различных PAC на табл. 1. Во всех примерах будем использовать евклидову метрику.

Таблица 1: Пример данных для Domain PAC, FD PAC и UCC PAC

A	B	C
1	10	25
3	15	1
4	17	24
8	11	24
4	27	1
6	3	17
9	9	27

Domain PAC

Рассмотрим Domain PAC на столбце A по области $D = (3, 5)$. Такая Domain PAC выполняется, например, с $\varepsilon = 1$ и $\delta = \frac{4}{7}$ (в $(D \pm 1)$ попадают все значения, кроме 1, 8 и 9).

FD PAC

Построим FD PAC $\{A\} \rightarrow \{B\}$ с $\Delta_A = 1$. Для этого нужно выбрать ε_A и δ .

Возьмём, к примеру, $\varepsilon_A = 5$. Нужно подобрать δ такую, что условие на правую часть будет выполнено для доли кортежей, большей или равной δ , среди кортежей, для которых выполнено условие на левую часть.

Условие на левую часть выполняется для следующих пар кортежей:

(1, 10) и (1, 10)	(3, 15) и (4, 17)	(8, 11) и (9, 9)
(3, 15) и (3, 15)	(4, 17) и (3, 15)	(9, 9) и (8, 11)
(4, 17) и (4, 17)	(3, 15) и (4, 27)	
(8, 11) и (8, 11)	(4, 27) и (3, 15)	
(4, 27) и (4, 27)	(4, 17) и (4, 27)	
(6, 3) и (6, 3)	(4, 27) и (4, 17)	
(9, 9) и (9, 9)		

Понятно, что для пар вида (t_i, t_i) , перечисленных в левом столбце, условие на правую часть выполнено при любом ε_A . Условие на пра-

вую часть при $\varepsilon_A = 5$ выполнено для следующих пар из двух правых столбцов:

$$\begin{array}{ll} (3, 15) \text{ и } (4, 17) & (8, 11) \text{ и } (9, 9) \\ (4, 17) \text{ и } (3, 15) & (9, 9) \text{ и } (8, 11) \end{array}$$

Таким образом, условие на левую часть выполнено для 15 пар кортежей, а условие на правую часть — только для 11 (7 из левого столбца и 4 — из правых двух). Значит, $\delta = \frac{11}{15}$ для $\varepsilon_A = 5$ и получаем следующую FD PAC:

$$\text{если } |t_i[A] - t_j[A]| \leq 1, \text{ то } Pr(|t_i[B] - t_j[B]| \leq 5) \geq \frac{11}{15}$$

USS PAC

Построим USS PAC на столбцах $\{ B, C \}$. Возьмём, например, $\varepsilon = 3$. Нужно подобрать δ такое, что доля “достаточно близких” пар кортежей среди всех возможных пар кортежей в таблице не меньше δ .

Расстояние между кортежами не превосходит трёх для следующих пар кортежей:

$$\begin{array}{ll} (10, 25) \text{ и } (10, 25) & (10, 25) \text{ и } (11, 24) \\ (15, 1) \text{ и } (15, 1) & (11, 24) \text{ и } (10, 25) \\ (17, 24) \text{ и } (17, 24) & (10, 25) \text{ и } (9, 27) \\ (11, 24) \text{ и } (11, 24) & (9, 27) \text{ и } (10, 25) \\ (27, 1) \text{ и } (27, 1) & \\ (3, 17) \text{ и } (3, 17) & \\ (9, 27) \text{ и } (9, 27) & \end{array}$$

Всего в таблице 7 кортежей, то есть 49 пар кортежей. “Достаточно близки” 11 из них. Таким образом, для $\varepsilon = 3$, $\delta = \frac{11}{49}$.

2.3. Алгоритм PAC-Man

В работе F. Korn и др. [5] описан алгоритм PAC-Man (PAC Manager), который выполняет две задачи для набора PAC:

1. выбирает значения ε^* и δ^* такие, что $\text{PAC}_{\varepsilon^*}^{\delta^*}$ выполняется;

2. отслеживает изменения в данных и сигнализирует о нарушениях РАС.

В данной работе рассматривается только выбор ε^* и δ^* , без проверки стабильности РАС.

Выбор значений ε^* и δ^*

Для каждого правила РАС-Man строит эмпирическую функцию распределения (Empirical Cumulative Distribution Function, ECDF). Для каждого δ_i выбираются ε_i такой, что $\Pr(\varepsilon < \varepsilon_i) \geq \delta_i$, то есть, эмпирическая вероятность, что ошибка приближения РАС меньше ε_i , не меньше δ_i .

Для Domain РАС и FD РАС наибольший интерес представляют РАС с меньшими ε и большими δ , так как они означают, что зависимость с небольшой погрешностью почти выполняется. Для USS РАС, наоборот, наиболее интересны РАС с большими ε и меньшими δ , так как такая РАС означает, что большинство значений достаточно различаются даже при большом допуске по расстоянию. Поэтому для Domain РАС и FD РАС рассматриваются только $\delta \geq \delta_{\min}$, а для USS РАС — только $\delta \leq \delta_{\max}$ для некоторых заданных $\delta_{\min}$ и $\delta_{\max}$.

Для выбора оптимальных значений ε^* и δ^* используется локтевой метод (elbow method) [6]. Обычно ECDF содержит “перелом” (“локоть”) в некоторой точке. Выбор $(\varepsilon^*, \delta^*)$ в этой точке будет оптимальным, так как дальнейшее увеличение ε не приведёт к значительному увеличению δ . Для нахождения точки “перелома” ε_s РАС-Man использует эвристический подход, основанный на поиске наиболее различающихся значений ε_i :

$$s = \operatorname{argmax}_{\delta_{\min} \leq \delta_i \leq \delta_{\max}} (\varepsilon_i - \varepsilon_{i-1})$$

Заметим, что данный подход неприменим для зависимостей, имеющих различные ε_l (или δ_l) для нескольких атрибутов. Поэтому в приведённой реализации рассматриваются только FD РАС с $\varepsilon_1 = \varepsilon_2 = \dots = \varepsilon$, а вместо UK РАС рассматриваются USS РАС.

3. Описание решения

Как уже отмечалось в предыдущей работе [7], алгоритм PAC-Man, описанный в разделе 2.3, вынесен в общий базовый класс PACVerifier, а классы, представляющие алгоритмы верификации конкретных вариантов РАС, должны предоставить метод, вычисляющий “эмпирические вероятности” — пары $(\varepsilon_i, \delta_i)$ такие, что выполняется $\text{PAC}_{\varepsilon_i}^{\delta_i}$.

Алгоритмы верификации FD РАС и USS РАС имеют много общего, поэтому был выделен промежуточный класс PairwisePACVerifier, представляющий алгоритм верификации РАС, работающий не с отдельными кортежами, а с парами кортежей. В классе PairwisePACVerifier используется шаблон проектирования Шаблонный метод [3] для реализации обобщённых операций. Виртуальные функции, реализованные в конкретных классах, которые вызывает Шаблонный метод, будем называть примитивными операциями.

Вычисление “эмпирических вероятностей”

Для вычисления “эмпирических вероятностей” используется Шаблонный метод CalculateEmpiricalProbabilities (листинг 1). Примитивные операции показаны в листинге синим цветом.

На вход CalculateEmpiricalProbabilities принимает отсортированный набор релевантных пар кортежей P , для которых вычислены расстояния по интересующим атрибутам. Для FD РАС P содержит пары кортежей, для которых выполнено условие на левую часть, а расстояния вычислены для атрибутов в правой части зависимости. Для USS РАС P содержит все возможные пары кортежей и расстояния по всем рассматриваемым атрибутам. При этом, в обоих случаях рассматриваются только пары кортежей (t_i, t_j) такие, что $i < j$, так как, в силу симметричности и невырожденности метрики, остальные пары избыточны.

CalculateEmpiricalProbabilities начинает с проверки и обработки “граничных” параметров: $\varepsilon_{\min}$, $\varepsilon_{\max}$, $\delta_{\min}$, $\delta_{\max}$ и n_δ .

Далее для каждого рассматриваемого значения δ_i вычисляется ε_i .

Для этого вычисляется m — количество пар, которое нужно взять, чтобы получить эмпирическую вероятность не меньше δ_i , и первые m пар выделяются в набор P' . Расстояние между кортежами для последней пары из P' берётся в качестве ε_i .

После этого выбирается наиболее подходящее (с точки зрения конкретного варианта РАС) значение δ_i , для которого выполняется РАС с тем же ε_i . При этом, примитивная операция **RefineDelta** работает с P' , а не с δ_i , так что после каждой итерации P' содержит ровно те пары, которые составляют δ_i .

Набор P отсортирован по возрастанию расстояний между кортежами, поэтому все описанные здесь манипуляции с P' сводятся к движению итератора, указывающего на конец P' .

Листинг 1. CalculateEmpiricalProbabilities

```

input  :  $P, \varepsilon_{\min}, \varepsilon_{\max}, \delta_{\min}, \delta_{\max}, n_\delta$ 
output :  $\{ (\delta_i, \varepsilon_i) \}$ 
1  $m_{\min} \leftarrow \text{GetNumPairs}(\delta_{\min})$ 
2  $m_{\max} \leftarrow \text{GetNumPairs}(\delta_{\max})$ 
3  $m_s \leftarrow \frac{m_{\max} - m_{\min}}{n_\delta - 1}$ 
4 for  $m \leftarrow m_{\min}$  to  $m_{\max}$  step  $m_s$  do
5   | Grow  $P$  to  $m$ 
6   |  $\varepsilon_i \leftarrow \text{last}(P').\text{distance}$ 
7   | RefineDelta( $P'$ )
8   |  $\delta_i \leftarrow \text{GetDelta}(\text{size}(P'))$ 
9 end
```

Примитивные операции

GetDelta, GetNumPairs

Вследствие того, что P содержит не все возможные пары, для определения вероятности δ_i по количеству пар m и наоборот, используются примитивные операции **GetDelta** и **GetNumPairs**.

В случае FD РАС $X \rightarrow Y$ набор P содержит те пары кортежей (t_i, t_j) , для которых $i < j$, а также выполнено условие на левую часть FD РАС:

$$\rho(t_i[A_l], t_j[A_l]) \leq \Delta_l \quad \forall A_l \in X$$

Обозначим за Γ множество всех пар кортежей, для которых выполнено условие на левую часть, $\Gamma' = \{ (t_i, t_j) \in \Gamma \mid i < j \}$, $|r|$ — общее количество кортежей в отношении. За σ обозначим подмножество Γ , состоящее из пар, для которых выполнено условие на правую часть FD PAC:

$$\rho(t_i[B_l], t_j[B_l]) \leq \varepsilon_l \quad \forall B_l \in Y$$

Таким образом,

$$\delta_i = \frac{|\sigma|}{|\Gamma|} = \frac{2m + |r|}{2|\Gamma'| + |r|}$$

$$m = \max \left\{ 0, \delta_i \cdot |\Gamma'| + \frac{|r| \cdot (\delta_i - 1)}{2} \right\}$$

В случае USS PAC набор P содержит все пары кортежей (t_i, t_j) , для которых $i < j$. Множество всех возможных пар кортежей обозначим за Γ . За σ обозначим множество пар, для которых выполнено условие USS PAC:

$$\rho(t_i[X], t_j[X]) \leq \varepsilon$$

Тогда

$$\delta_i = \frac{|\sigma|}{|\Gamma|} = \frac{2m + |r|}{|r|^2}$$

$$m = \max \left\{ 0, \frac{|r| \cdot (\delta_i \cdot |r| - 1)}{2} \right\}$$

RefineDelta

Из определений FD PAC и USS PAC видно, что, при фиксированном ε , для FD PAC предпочтительным будет большее δ , а для USS PAC — меньшее. При этом, как уже упоминалось ранее, значения в P отсортированы по возрастанию расстояния между кортежами. Исходя из этого, примитивная операция **RefineDelta** в случае FD PAC сдвигает правый край P' вправо, пока расстояние для последней пары в P' равно ε_i . В случае USS PAC правый край P' сдвигается влево.

Вычисление исключений

Алгоритмы верификации в DESBORDANTE вычисляют “исключения” — наборы кортежей (или пар кортежей), из-за которых зависимость не выполняется ли выполняется с худшими параметрами.

Для FD РАС $X \rightarrow Y$ каждой паре кортежей (t_1, t_2) сопоставим значение $e(t_1, t_2) = \max_{B_l \in Y} \{ \rho(t_1[B_l], t_2[B_l]) \}$. За Γ обозначим множество пар, для которых выполнено условие на левую часть FD РАС. Тогда исключением с параметрами $\varepsilon_1, \varepsilon_2$ будет множество $\{ (t_1, t_2) \in \Gamma \mid \varepsilon_1 < e(t_1, t_2) \leq \varepsilon_2 \}$.

Для USS РАС на множестве атрибутов X каждой паре кортежей (t_1, t_2) сопоставим значение $e(t_1, t_2) = \rho(t_1[X], t_2[X])$. Тогда исключение с параметрами $\varepsilon_1, \varepsilon_2$ будет состоять из пар, для которых $\varepsilon_1 < e \leq \varepsilon_2$.

Важно отметить, что вероятностные параметры РАС не влияют на исключения, в то время как параметры, задающие правило — влияют.

Для вычисления исключений бинарным поиском по P находятся первая пара $(t_1^{(1)}, t_2^{(1)})$, для которой $e(t_1^{(1)}, t_2^{(1)}) > \varepsilon_1$, и первая пара $(t_1^{(2)}, t_2^{(2)})$, для которой $e(t_1^{(2)}, t_2^{(2)}) > \varepsilon_2$. Исключение состоит из пар между $(t_1^{(1)}, t_2^{(1)})$ и $(t_1^{(2)}, t_2^{(2)})$, а также “симметричных” к ним: для каждой пары (t_i, t_j) , пара (t_j, t_i) также входит в исключение.

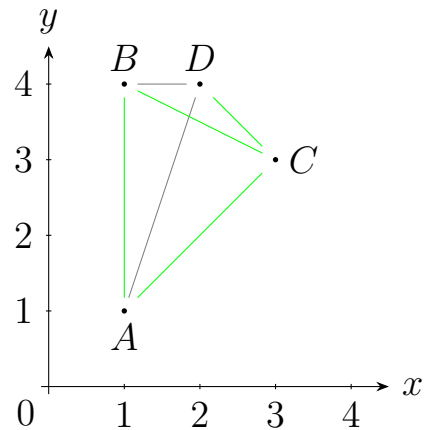
Пример

Рис. 1: Пример исключений для FD РАС и USS РАС

(a) Данные для примера

Метка	x	y
A	1	1
B	1	4
C	3	3
D	2	4

(b) Иллюстрация к примеру



На рис. 1 приведена таблица с координатами некоторых точек, а

также показано расположение этих точек на плоскости.

Рассмотрим USS PAC на атрибутах $\{x, y\}$ с евклидовой метрикой. Исключения с параметрами $\varepsilon_1 = 1$, $\varepsilon_2 = 3$ будут состоять из следующих пар (для наглядности приведены метки точек, а не номера кортежей): $\{(A, C), (B, C), (C, D)\}$, а также “симметричных” к ним: $\{(C, A), (C, B), (D, C)\}$. На рис. 1b зелёным цветом показаны пары кортежей, расстояния между которыми попадают в диапазон, задаваемый параметрами $\varepsilon_1, \varepsilon_2$, а серым — все остальные.

Для FD PAC исключения вычисляются аналогично. Возьмём FD PAC $\{\text{Метка}\} \rightarrow \{x, y\}$. На атрибутах x и y возьмём евклидову метрику. На атрибуте Метка введём дискретную метрику: $\rho(a, b) = 0$, если $a = b$, и $\rho(a, b) = 1$ иначе. Возьмём $\Delta_{\text{Метка}} = 0$.

Для такой FD PAC исключения с параметрами $\varepsilon_1 = 1$, $\varepsilon_2 = 3$ будут состоять из тех же пар кортежей, что и исключения для USS PAC в предыдущем примере.

Кластеризация значений

Исключения для FD PAC и USS PAC состоят из большого числа пар кортежей. Для дальнейшей обработки полученных исключений может быть полезно провести их кластеризацию — выделить группы кортежей, близких друг к другу. На кластеры можно накладывать различные ограничения, такие как максимальный размер кластера, или максимальное количество кластеров.

Рассмотрим один из возможных способов ввести такие ограничения — зафиксируем количество кластеров и максимальный диаметр кластера. Эту задачу можно формализовать следующим образом [4, задача MS9]:

Дан полный взвешенный граф $G = (V, E)$ с неотрицательными весами рёбер, и два числа $1 \leq k \leq |V|$ и $d \geq 0$. Требуется разбить G на k подграфов так, чтобы все метки на рёбрах каждого подграфа не превосходили d .

Известно, что эта задача является NP-полной даже для случая, когда $k = 3$, и все метки на рёбрах принимают только значения 0 и 1 [4].

Поскольку это не относится напрямую к решаемой задаче верификации PAC, было принято решение не реализовывать кластеризацию значений, а позволить пользователю самостоятельно выбрать, какой метод решения данной задачи использовать. Кроме того, существуют готовые алгоритмы для решения данной задачи. Например, популярная Python-библиотека `scikit-learn` предоставляет алгоритмы как для точного, так и для приближённого решения задачи кластеризации².

Интеграция алгоритмов в DESBORDANTE

В DESBORDANTE каждый алгоритм представляется классом, унаследованным от `Algorithm`. Алгоритм PAC-Man реализован в классе `PACVerifier`, от которого наследуются классы, представляющие алгоритмы верификации для конкретных типов PAC. Как уже упоминалось ранее в данном разделе, часть общей функциональности алгоритмов верификации FD PAC и USS PAC вынесена в класс `PairwisePACVerifier`.

Диаграмма классов, участвующих в верификации PAC, показана на рис. 2. Синим цветом показаны классы и модули, ранее присутствовавшие в DESBORDANTE, красным — реализованные в ходе предыдущей работы, жёлтым — реализованные в ходе данной работы.

²https://contrib.scikit-learn.org/radius_clustering/index.html

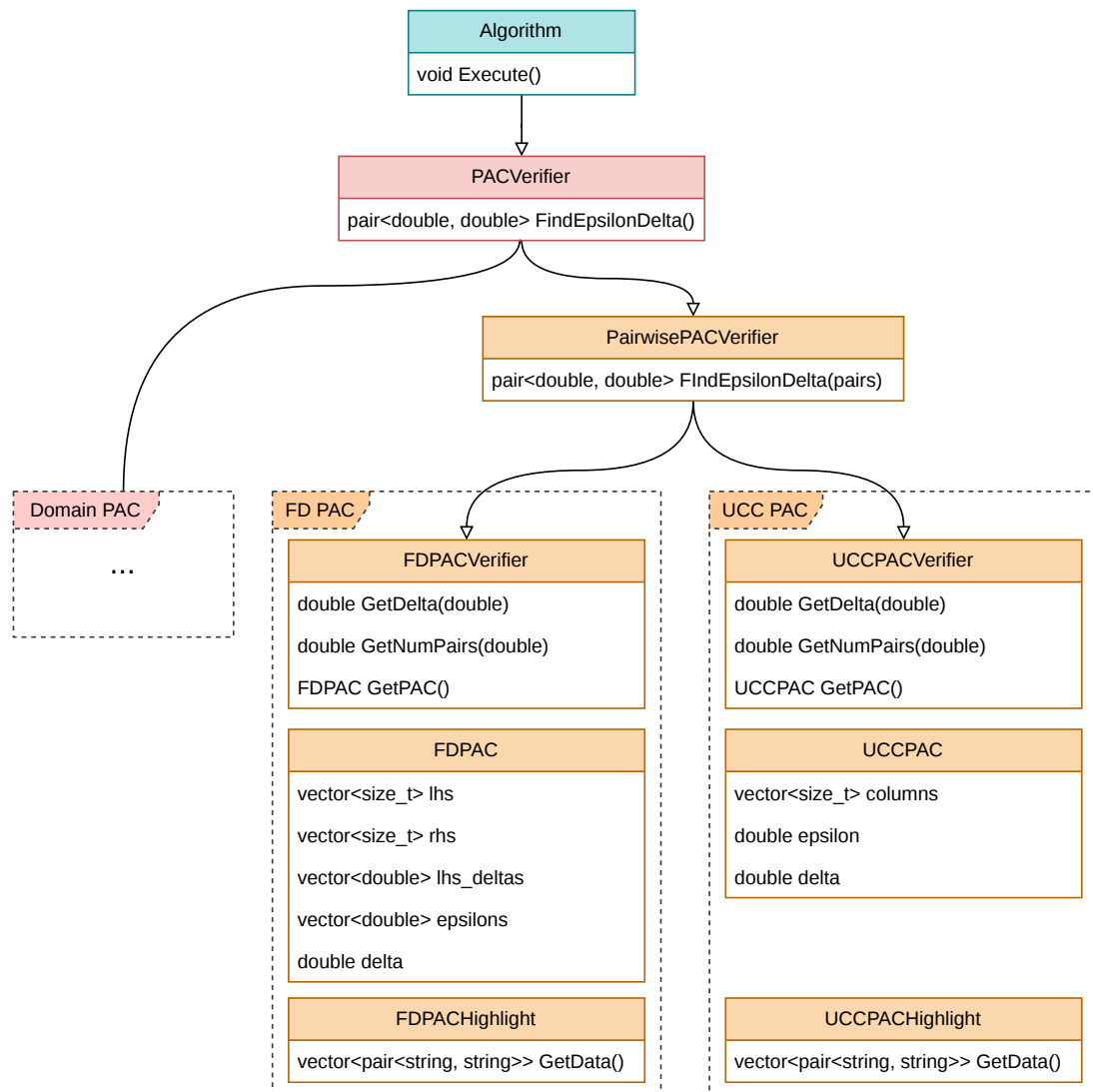


Рис. 2: Диаграмма классов верификации PAC

4. Апробация

Основная задача эксперимента — оценить и сравнить между собой масштабируемость алгоритмов верификации Domain PAC, FD PAC и UCC PAC. Таким образом, были поставлены следующие исследовательские вопросы:

RQ1. Как время работы трёх алгоритмов зависит от количества строк в таблице?

RQ2. Как время работы алгоритмов зависит от количества столбцов,

участвующих в PAC (арности PAC)?

RQ3. Каково соотношение времени работы различных стадий алгоритмов?

В данном разделе приведены только графики, иллюстрирующие ответы на исследовательские вопросы. Полные результаты измерений, их анализ, а также команды, при помощи которых производилась подготовка к измерениям, доступны на GitHub³. Измерения проводились от 10 до 30 раз, для различных RQ и алгоритмов.

Для проведения измерений использовалась система со следующими характеристиками:

- Ubuntu 24.04 LTS;
- AMD Zen 4 @2.396GHz, 4 vCPU;
- оперативная память: 32 GiB.

Измерения проводились на следующих таблицах:

1. `iowa`⁴ — содержит 24 столбца с естественными данными различных типов. Для экспериментов использовались части таблицы `iowa` с различным количеством строк. Размеры этих таблиц приведены в табл. 2.
2. `uniform-wide`⁵ — содержит 50 000 строк и 20 столбцов с синтетическими данными. Все столбцы содержат только вещественные числа. Размер таблицы — 17.3 МБ.

Во всех экспериментах для столбцов, содержащих числа, использовалась евклидова метрика, для столбцов, содержащих строки, — расстояние Левенштейна.

³<https://github.com/p-senichenkov/Desbordante/tree/PAC/validation/common/triple-benchmark>

⁴<https://github.com/Desbordante/desbordante-data>

⁵https://github.com/p-senichenkov/Desbordante/blob/PAC/validation/common/triple-benchmark/test_input_data/pac_data/uniform-wide.csv

Таблица 2: Размеры таблиц `iowa` с различным количеством строк.

Количество строк, тыс.	Размер, МБ
5	1.0
10	2.1
20	4.2
50	10.5
100	21.0
200	41.9

RQ1

Для ответа на **RQ1** был проведён ряд измерений на таблицах `iowa` с различным количеством строк. Все измерения проводились для РАС арности 4 (для FD РАС — по 2 атрибута в левой и правой частях), на столбцах, содержащих различные типы данных. Результаты измерений приведены на рис. 3.

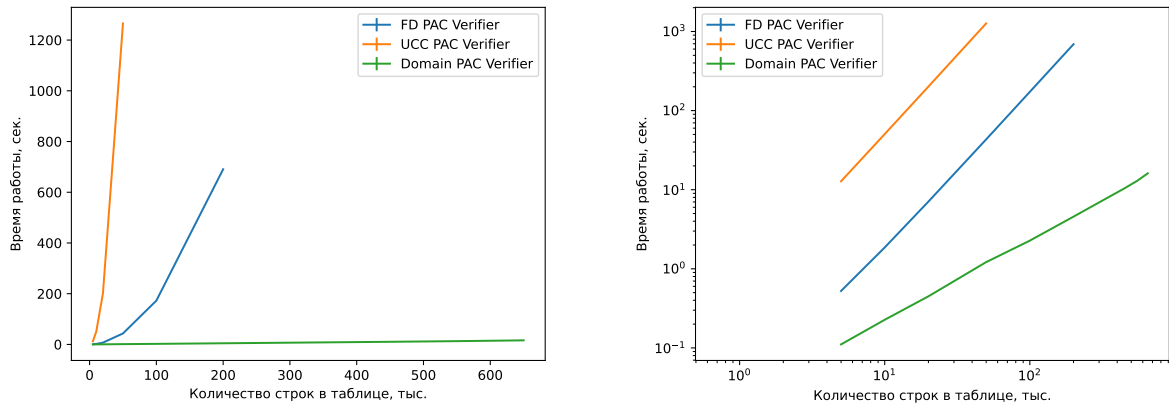


Рис. 3: Результаты измерения масштабируемости алгоритмов верификации РАС от количества строк в таблице в линейной и логарифмической шкалах

Асимптотическая сложность алгоритма верификации Domain РАС — $O(n \cdot \log n)$ от количества строк в таблице (нужно вычислить и отсортировать расстояния от каждого кортежа в таблице). Для алгоритмов верификации FD РАС и UCC РАС асимптотическая сложность $O(n^2 \cdot \log n)$, так как нужно вычислить и отсортировать все попарные расстояния между кортежами. При этом, алгоритм для FD РАС работа-

ет с относительно небольшим (на естественных данных) подмножеством пар кортежей, для которых выполнено условие на левую часть, в то время как алгоритм для USS PAC должен рассматривать все возможные пары кортежей. Можно видеть, что результаты, полученные в ходе эксперимента, соответствуют этим теоретическим выводам.

RQ2

Для ответа на **RQ2** были проведены эксперименты на таблице **uniform-wide** для PAC различной арности. Результаты экспериментов приведены на рис. 4.

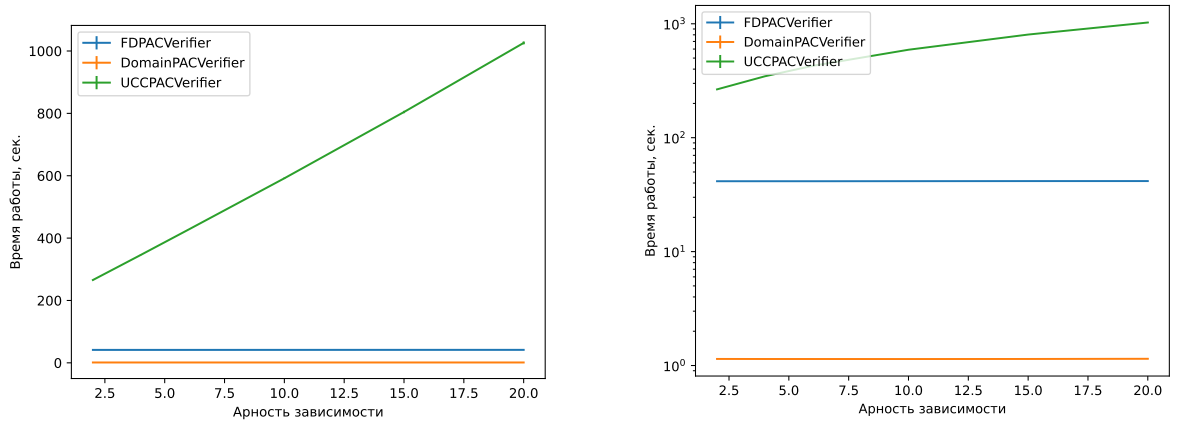


Рис. 4: Результаты измерения масштабируемости алгоритмов верификации PAC от арности PAC в линейной и логарифмической шкалах

Для Domain PAC вычисляется расстояние до области один раз для каждого кортежа в таблице.

Для FD PAC необходимо вычислить расстояние между значениями по каждому из атрибутов в левой части зависимости для каждой пары кортежей, а затем — по каждому из атрибутов в правой части для тех пар, для которых левые части оказались достаточно близки. При этом, данные в таблице **uniform-wide** распределены равномерно, а значит, можно считать, что доля таких пар пренебрежимо мала.

Кроме того, условие на левую часть FD PAC проверяются “лениво” — до тех пор, пока не встретится атрибут, на котором условие не выполнено. Значит, на естественных данных количество проверок условия для

каждой пары кортежей растёт медленнее, чем линейно от арности FD PAC.

Для USS PAC нельзя отсечь никакие пары кортежей, поэтому необходимо вычислить попарные расстояния для всех кортежей в таблице. Кроме того, в FD PAC используются расстояния по одиночным атрибутам, а в USS PAC кортеж целиком рассматривается как единое значение. Вычисление таких метрик часто сложнее, чем вычисление метрик по отдельным атрибутам.

RQ3

Для ответа на **RQ3** были проведены эксперименты на таблице *iowa*, содержащей 50 000 строк, на тех же атрибутах, что и в **RQ1**. Было измерено время работы основных стадий работы алгоритмов:

1. **Чтение таблицы** из csv-файла, определение типов данных в столбцах и конвертация значений.
2. **Объединение значений в кортежи**. Таблица с входными данными в DESBORDANTE представляется как набор столбцов, поэтому алгоритмам верификации PAC необходимо предварительно представить входные данные в виде набора кортежей.
3. **Вычисление расстояний**. На этом этапе вычисляются расстояния от каждого кортежа до домена (для Domain PAC) или попарные расстояния между кортежами (для FD PAC и USS PAC).
4. **Сортировка** вычисленных на предыдущем этапе **расстояний**.
5. Поиск оптимальных значений ε^* , δ^* с помощью алгоритма **PAC-Man**.
6. **Вычисление исключений**. В данном эксперименте для всех алгоритмов вычислялись исключения с параметрами $\varepsilon_1 = 1$, $\varepsilon_2 = 10$.

Эксперименты проводились на РАС, состоящих из четырёх атрибутов (два в левой части, и два — в правой для FD РАС).

Результаты измерений приведены на рис. 5. Можно сделать следующие выводы:

- Для алгоритма верификации Domain РАС чтение таблицы занимает гораздо больше времени, чем остальные стадии. В предыдущей работе [7] этот вопрос изучался более подробно, и был сделан вывод, что для больших наборов данных чтение таблицы является единственной значимой стадией работы алгоритма.
- Для двух других алгоритмов основное время затрачивается на вычисление расстояний между кортежами. Это обусловлено тем, что для Domain РАС необходимо вычислить только расстояния от каждого кортежа до области, в то время как для FD РАС и USS РАС необходимо вычислять попарные расстояния между кортежами, а значит время работы этой стадии зависит от количества строк в таблице квадратично.
- Для алгоритма верификации USS РАС также много времени затрачивается на сортировку расстояний между кортежами. Это происходит из-за того, что в USS РАС нет ограничения на левую часть зависимости, а значит, в отличие от FD РАС, алгоритм для USS РАС работает с попарными расстояниями между всеми кортежами в таблице, а не с небольшим (для естественных данных) их подмножеством.

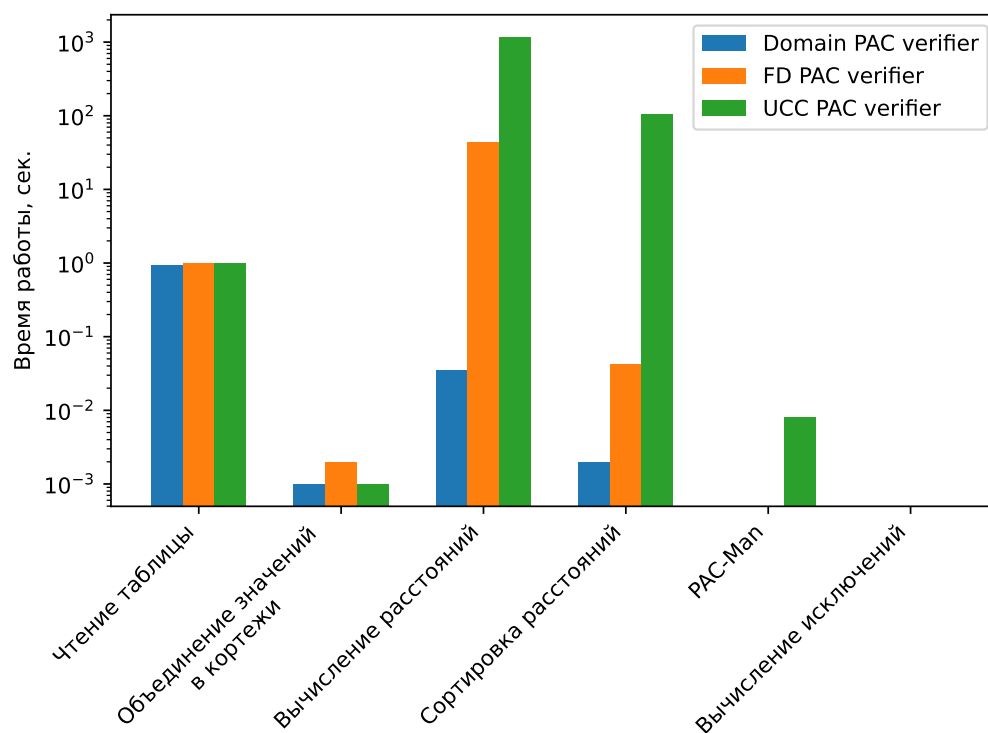


Рис. 5: Распределение времени работы по стадиям алгоритмов

5. Заключение

В ходе данной работы была добавлена поддержка верификации функциональных вероятностных приближённых ограничений и вероятностных приближённых ограничений по уникальной комбинации столбцов. Результаты работы:

1. реализованы алгоритмы верификации FD PAC и UCC PAC;
2. разработаны модульные тесты и соответствующий набор данных для тестов;
3. реализованы Python-привязки к алгоритмам верификации;
4. созданы примеры использования;
5. проведена апробация и сравнение алгоритмов верификации PAC.

Исходный код доступен на GitHub⁶. Изменения находятся на стадии рассмотрения.

⁶<https://github.com/Desbordante/desbordante-core/pull/706>,
<https://github.com/Desbordante/desbordante-core/pull/738>

<https://github.com/Desbordante/>

Список литературы

- [1] [Data Profiling](#) / Ziawasch Abedjan, Lukasz Golab, Felix Naumann, Thorsten Papenbrock. — Springer Cham, 2018. — Nov. — ISBN: [978-3-031-00737-8](#).
- [2] [Desbordante: from benchmarking suite to high-performance science-intensive data profiler](#) / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: <https://doi.org/10.1145/3703323.3703725>.
- [3] Design Patterns: Elements of Reusable Object-Oriented Software / R. Johnson, E. Gamma, J. Vlissides, R. Helm. Addison-Wesley professional computing series. — Addison-Wesley, 2005. — ISBN: [9780201633610](#).
- [4] Garey Michael R, Johnson David S. Computers and intractability. — wh freeman New York, 2002. — Vol. 29.
- [5] Korn Flip, Muthukrishnan S., Zhu Yunyue. Checks and balances: Monitoring data quality problems in network traffic databases // Proceedings 2003 VLDB Conference / Elsevier. — 2003. — P. 536–547.
- [6] Thorndike Robert L. Who Belongs in the Family? // [Psychometrika](#). — 1953. — Vol. 18, no. 4. — P. 267–276.
- [7] Пётр Сениченков. Реализация алгоритма верификации вероятностных приближённых ограничений по области в “Desbordante”. — 2026. — URL: <https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/DomainPACverifier-SenichenkovPyotr-2025autumn.pdf>.